AI ASSISTANT CODING - ASSIGNMENT-17.3

Name: Valaboju Harshavardhanu

Roll No: 2303A52354 **Batch:** 41

Task 1: I-Employee Dataset Cleaning

Prompt used:

Create a Python script in Google Colab to clean an employee dataset. Create a dataset manually using pandas. Handle missing values in salary using median. missing values in designation using mode. Handle missing values in experience using mean. Remove duplicate employee IDs. Convert all job titles (designation) to lowercase. Encode categorical columns (designation, employment_type) using LabelEncoder. Display the cleaned dataset.

Code:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# 1. Create a sample employee dataset
data = {
    'employee_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 1, 11],
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy', 'Alic
    'salary': [70000, 60000, None, 90000, 55000, 72000, 68000, None, 85000, 71000, 70000, 63000],
    'designation': ['Software Engineer', 'Data Scientist', 'Project Manager', 'Software Engineer', 'UX D
    'experience': [5, 3, 8, 6, 2, None, 4, 7, 9, 5, 5, 3],
    'employment_type': ['Full-time', 'Full-time', 'Contract', 'Full-time', 'Part-time', 'Full-time', 'Fu
}
df = pd.DataFrame(data)
print("Original Dataset:")
display(df)
print("\nInfo before cleaning:")
df.info()

# 2. Handle missing values
# Handle missing values in salary using median
median_salary = df['salary'].median()
df['salary'].fillna(median_salary, inplace=True)

# Handle missing values in designation using mode
mode_designation = df['designation'].mode()[0]
```

```python
# Handle missing values in designation using mode
mode_designation = df['designation'].mode()[0]
df['designation'].fillna(mode_designation, inplace=True)

# Handle missing values in experience using mean
mean_experience = df['experience'].mean()
df['experience'].fillna(mean_experience, inplace=True)

# 3. Remove duplicate employee IDs
df.drop_duplicates(subset=['employee_id'], inplace=True)

# 4. Convert all job titles (designation) to lowercase
df['designation'] = df['designation'].str.lower()

# 5. Encode categorical columns (designation, employment_type) using LabelEncoder
le_designation = LabelEncoder()
df['designation_encoded'] = le_designation.fit_transform(df['designation'])

le_employment_type = LabelEncoder()
df['employment_type_encoded'] = le_employment_type.fit_transform(df['employment_type'])

# Display the cleaned dataset
print("\nCleaned Dataset:")
display(df)
print("\nInfo after cleaning:")
df.info()
```

Original Dataset (Before Preprocessing):

Original Dataset:

	employee_id	name	salary	designation	experience	employment_type
0	1	Alice	70000.0	Software Engineer	5.0	Full-time
1	2	Bob	60000.0	Data Scientist	3.0	Full-time
2	3	Charlie	NaN	Project Manager	8.0	Contract
3	4	David	90000.0	Software Engineer	6.0	Full-time
4	5	Eve	55000.0	UX Designer	2.0	Part-time
5	6	Frank	72000.0	Data Scientist	NaN	Full-time
6	7	Grace	68000.0	None	4.0	Full-time
7	8	Heidi	NaN	Marketing Specialist	7.0	Contract
8	9	Ivan	85000.0	Project Manager	9.0	Full-time
9	10	Judy	71000.0	Software Engineer	5.0	Full-time
10	1	Alice	70000.0	Software Engineer	5.0	Full-time
11	11	Kevin	63000.0	QA Engineer	3.0	Contract

Output (After Preprocessing):

	employee_id	name	salary	designation	experience	employment_type	designation_encoded	employment_type_encoded
0	1	Alice	70000.0	software engineer	5.000000	Full-time	4	1
1	2	Bob	60000.0	data scientist	3.000000	Full-time	0	1
2	3	Charlie	70000.0	project manager	8.000000	Contract	2	0
3	4	David	90000.0	software engineer	6.000000	Full-time	4	1
4	5	Eve	55000.0	ux designer	2.000000	Part-time	5	2
5	6	Frank	72000.0	data scientist	5.181818	Full-time	0	1
6	7	Grace	68000.0	software engineer	4.000000	Full-time	4	1
7	8	Heidi	70000.0	marketing specialist	7.000000	Contract	1	0
8	9	Ivan	85000.0	project manager	9.000000	Full-time	2	1
9	10	Judy	71000.0	software engineer	5.000000	Full-time	4	1
11	11	Kevin	63000.0	qa engineer	3.000000	Contract	3	0

```
Info after cleaning:
```

Justification:

Cleaning the employee dataset ensures accurate HR analysis by handling missing values in salary, designation, and experience to avoid incorrect insights.Removing duplicate IDs and standardizing and encoding data makes the dataset consistent and ready for machine learning

Task 2: Student Performance Data Preprocessing

Prompt used:

Create a Python script to preprocess student performance data. Create dataset manually using pandas. Convert exam_date column into datetime format. Extract new features: month, day, and semester. Normalize total_marks using MinMaxScaler. Detect and remove outliers using IQR method. Display the transformed dataset.

Code:

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# 1. Create a sample student performance dataset
data = {
    'student_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15],
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy', 'Kevin', 'L
    'exam_date': ['2023-03-15', '2023-09-20', '2023-04-10', '2023-10-05', '2023-05-22', '2023-01-18', '2023-07
    'total_marks': [85, 72, 90, 68, 95, 78, 50, 88, 150, 65, 80, 75, 92, 70, 10],
    'course': ['Math', 'Physics', 'Math', 'Chemistry', 'Physics', 'Biology', 'Math', 'Physics', 'Math', 'Chemi
}
df_students = pd.DataFrame(data)

print("Original Dataset:")
display(df_students)
print("\nInfo before cleaning:")
df_students.info()

# 2. Convert 'exam_date' column into datetime format
df_students['exam_date'] = pd.to_datetime(df_students['exam_date'])

# 3. Extract new features: month, day, and semester
df_students['exam_month'] = df_students['exam_date'].dt.month
df_students['exam_day'] = df_students['exam_date'].dt.day
```

```python
# Define a simple semester logic:
# Spring: Jan-May, Summer: Jun-Aug, Fall: Sep-Dec
def get_semester(month):
    if 1 <= month <= 5:
        return 'Spring'
    elif 6 <= month <= 8:
        return 'Summer'
    else:
        return 'Fall'

df_students['semester'] = df_students['exam_month'].apply(get_semester)

# 4. Normalize 'total_marks' using MinMaxScaler
scaler = MinMaxScaler()
df_students['total_marks_normalized'] = scaler.fit_transform(df_students[['total_marks']])

# 5. Detect and remove outliers using IQR method
Q1 = df_students['total_marks'].quantile(0.25)
Q3 = df_students['total_marks'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outlier_mask = (df_students['total_marks'] < lower_bound) | (df_students['total_marks'] > upper_bound)
df_students_cleaned = df_students[~outlier_mask]

print("\nCleaned Dataset (outliers removed, new features, normalized marks):")
display(df_students_cleaned)
print("\nInfo after cleaning:")
df_students_cleaned.info()
```

Original Dataset (Before Preprocessing):

Original Dataset:

	student_id	name	exam_date	total_marks	course
0	1	Alice	2023-03-15	85	Math
1	2	Bob	2023-09-20	72	Physics
2	3	Charlie	2023-04-10	90	Math
3	4	David	2023-10-05	68	Chemistry
4	5	Eve	2023-05-22	95	Physics
5	6	Frank	2023-01-18	78	Biology
6	7	Grace	2023-07-30	50	Math
7	8	Heidi	2023-02-28	88	Physics
8	9	Ivan	2023-11-12	150	Math
9	10	Judy	2023-06-01	65	Chemistry
10	11	Kevin	2023-03-01	80	Biology
11	12	Liam	2023-09-15	75	Math
12	13	Mia	2023-04-20	92	Physics
13	14	Noah	2023-10-10	70	Chemistry
14	15	Olivia	2023-05-05	10	Biology

Output (After Preprocessing):

Cleaned Dataset (outliers removed, new features, normalized marks):

	student_id	name	exam_date	total_marks	course	exam_month	exam_day	semester	total_marks_normalized
0	1	Alice	2023-03-15	85	Math	3	15	Spring	0.535714
1	2	Bob	2023-09-20	72	Physics	9	20	Fall	0.442857
2	3	Charlie	2023-04-10	90	Math	4	10	Spring	0.571429
3	4	David	2023-10-05	68	Chemistry	10	5	Fall	0.414286
4	5	Eve	2023-05-22	95	Physics	5	22	Spring	0.607143
5	6	Frank	2023-01-18	78	Biology	1	18	Spring	0.485714
6	7	Grace	2023-07-30	50	Math	7	30	Summer	0.285714
7	8	Heidi	2023-02-28	88	Physics	2	28	Spring	0.557143
9	10	Judy	2023-06-01	65	Chemistry	6	1	Summer	0.392857
10	11	Kevin	2023-03-01	80	Biology	3	1	Spring	0.500000
11	12	Liam	2023-09-15	75	Math	9	15	Fall	0.464286
12	13	Mia	2023-04-20	92	Physics	4	20	Spring	0.585714
13	14	Noah	2023-10-10	70	Chemistry	10	10	Fall	0.428571

Info after cleaning:

Justification:

Preprocessing student data helps in understanding academic patterns more effectively. Converting exam dates allows extraction of useful features like month and semester. Normalizing marks ensures all values are on the same scale, improving model performance. Removing outliers prevents extreme values from affecting analysis, making predictions more reliable.

Task 3: Student Health Data Preparation

Prompt used:

Create a Python script for student health dataset preparation. Create dataset manually using pandas. Handle missing values in height, weight, BMI using mean. Encode categorical features (medical_condition, fitness_status) using one-hot encoding. Scale numerical features using StandardScaler. Split dataset into training and testing sets (80:20). Display processed dataset and shapes of train/test sets.

Code:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
import numpy as np

# 1. Create a sample student health dataset
data = {
    'student_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'age': [18, 19, 18, 20, 19, 18, 20, 19, 20, 18],
    'height': [170, 165, np.nan, 175, 168, 172, 180, 170, 173, np.nan],
    'weight': [65, 60, 70, np.nan, 62, 68, 75, 66, np.nan, 63],
    'medical_condition': ['None', 'Asthma', 'None', 'Allergies', 'None', 'None', 'Asthma', 'None', 'Allergies', 'None'],
    'fitness_status': ['Good', 'Average', 'Good', 'Excellent', 'Average', 'Good', 'Excellent', 'Average', 'Good', 'Average'],
    'BMI': [22.5, 22.0, 23.0, 23.5, np.nan, 23.0, 23.1, 22.9, 24.0, 22.8]
}
df_health = pd.DataFrame(data)

print("Original Dataset:")
display(df_health)
print("\nInfo before preprocessing:")
df_health.info()

# Separate features into numerical and categorical
numerical_cols = ['age', 'height', 'weight', 'BMI']
categorical_cols = ['medical_condition', 'fitness_status']
```

```python
# 3. Encode categorical features using one-hot encoding
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
encoded_features = encoder.fit_transform(df_health[categorical_cols])
encoded_df = pd.DataFrame(encoded_features, columns=encoder.get_feature_names_out(categorical_cols), index=df_health.index)

# Combine numerical (imputed) and encoded categorical features
df_processed = pd.concat([df_health[numerical_cols], encoded_df], axis=1)

# 4. Scale numerical features using StandardScaler
scaler = StandardScaler()
df_processed[numerical_cols] = scaler.fit_transform(df_processed[numerical_cols])

print("\nProcessed Dataset (missing values handled, categorical encoded, numerical scaled):")
display(df_processed)
print("\nInfo after preprocessing:")
df_processed.info()

# 5. Split dataset into training and testing sets (80:20)
X = df_processed
y = df_health['fitness_status'] # Using fitness_status as a sample target for splitting

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("\nShapes of training and testing sets:")
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

Original Dataset (Before Preprocessing):

Original Dataset:

	student_id	age	height	weight	medical_condition	fitness_status	BMI
0	1	18	170.0	65.0	None	Good	22.5
1	2	19	165.0	60.0	Asthma	Average	22.0
2	3	18	NaN	70.0	None	Good	23.0
3	4	20	175.0	NaN	Allergies	Excellent	23.5
4	5	19	168.0	62.0	None	Average	NaN
5	6	18	172.0	68.0	None	Good	23.0
6	7	20	180.0	75.0	Asthma	Excellent	23.1
7	8	19	170.0	66.0	None	Average	22.9
8	9	20	173.0	NaN	Allergies	Good	24.0
9	10	18	NaN	63.0	None	Average	22.8

Output (After Preprocessing):

Processed Dataset (missing values handled, categorical encoded, numerical scaled):

	age	height	weight	BMI	medical_condition_Allergies	medical_condition_Asthma	medical_condition_None	fitness_status_Average	fitness_status_Excellent	fitness_status_Good
0	-1.083473	-0.425464	-0.278757	-0.945112	0.0	0.0	1.0	0.0	0.0	1.0
1	0.120386	-1.734584	-1.517675	-1.934183	0.0	1.0	0.0	1.0	0.0	0.0
2	-1.083473	0.000000	0.960162	0.043959	0.0	0.0	1.0	0.0	0.0	1.0
3	1.324244	0.883656	0.000000	1.033029	1.0	0.0	0.0	0.0	1.0	0.0
4	0.120386	-0.949112	-1.022108	0.000000	0.0	0.0	1.0	1.0	0.0	0.0
5	-1.083473	0.098184	0.464594	0.043959	0.0	0.0	1.0	0.0	0.0	1.0
6	1.324244	2.192777	2.199081	0.241773	0.0	1.0	0.0	0.0	1.0	0.0
7	0.120386	-0.425464	-0.030973	-0.153855	0.0	0.0	1.0	1.0	0.0	0.0
8	1.324244	0.360008	0.000000	2.022100	1.0	0.0	0.0	0.0	0.0	1.0
9	-1.083473	0.000000	-0.774324	-0.351670	0.0	0.0	1.0	1.0	0.0	0.0

Info after preprocessing:

Justification:

Preparing health data is essential for building accurate predictive models. Missing values in height, weight, and BMI are handled to avoid incomplete records. Encoding categorical data makes it usable for machine learning algorithms. Scaling ensures all features contribute equally, and splitting the dataset helps evaluate model performance effectively

Task 4: Real-Time Application: Ride-Sharing Data Cleaning

Prompt used:

Create a Python script in Google Colab to clean ride-sharing data. Create dataset manually using pandas. Standardize pickup and drop locations (lowercase and remove spaces) .Fill missing fare values using median. Remove duplicate rows. Normalize driver_rating using MinMaxScaler. Display cleaned dataset. Generate a summary (shape and missing values).

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# 1. Create a sample ride-sharing dataset
data = {
    'ride_id': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 101],
    'pickup_location': [' Downtown ', ' Airport', 'Suburbs', 'Downtown', '  City Center  ', 'Airport', 'Suburbs', 'Dou
    'dropoff_location': ['Uptown', 'Hotel', 'Downtown', 'Station', '  Residential  ', 'Uptown', 'Hotel', 'Downtown',
    'fare': [15.5, 25.0, 12.0, np.nan, 18.0, 26.5, 13.0, 16.0, np.nan, 20.0, 15.5],
    'driver_rating': [4.8, 3.5, 4.9, 4.2, 5.0, 4.1, 3.9, 4.7, 4.5, 4.0, 4.8],
    'trip_distance_miles': [5.2, 12.1, 3.5, 6.0, 7.8, 11.5, 4.0, 5.5, 6.5, 8.0, 5.2]
}
df_rides = pd.DataFrame(data)

print("Original Dataset:")
display(df_rides)
print("\nInfo before cleaning:")
df_rides.info()

# 2. Standardize pickup and drop locations (lowercase and remove spaces)
df_rides['pickup_location'] = df_rides['pickup_location'].str.lower().str.strip()
df_rides['dropoff_location'] = df_rides['dropoff_location'].str.lower().str.strip()

# 3. Fill missing fare values using median
median_fare = df_rides['fare'].median()
df_rides['fare'].fillna(median_fare, inplace=True)
```

```python
# 4. Remove duplicate rows
df_rides.drop_duplicates(inplace=True)

# 5. Normalize driver_rating using MinMaxScaler
scaler = MinMaxScaler()
df_rides['driver_rating_normalized'] = scaler.fit_transform(df_rides[['driver_rating']])

print("\nCleaned Dataset:")
display(df_rides)

print("\nSummary of cleaned dataset:")
print(f"Shape of cleaned dataset: {df_rides.shape}")
print("Missing values after cleaning:")
print(df_rides.isnull().sum())
```

Original Dataset (Before Preprocessing):

Original Dataset:

	ride_id	pickup_location	dropoff_location	fare	driver_rating	trip_distance_miles
0	101	Downtown	Uptown	15.5	4.8	5.2
1	102	Airport	Hotel	25.0	3.5	12.1
2	103	Suburbs	Downtown	12.0	4.9	3.5
3	104	Downtown	Station	NaN	4.2	6.0
4	105	City Center	Residential	18.0	5.0	7.8
5	106	Airport	Uptown	26.5	4.1	11.5
6	107	Suburbs	Hotel	13.0	3.9	4.0
7	108	Downtown	Downtown	16.0	4.7	5.5
8	109	City Center	Station	NaN	4.5	6.5
9	110	Airport	Residential	20.0	4.0	8.0
10	101	Downtown	Uptown	15.5	4.8	5.2

Output (After Preprocessing):

df_rides['fare'].fillna(median_fare, inplace=True)

	ride_id	pickup_location	dropoff_location	fare	driver_rating	trip_distance_miles	driver_rating_normalized
0	101	downtown	uptown	15.5	4.8	5.2	0.866667
1	102	airport	hotel	25.0	3.5	12.1	0.000000
2	103	suburbs	downtown	12.0	4.9	3.5	0.933333
3	104	downtown	station	16.0	4.2	6.0	0.466667
4	105	city center	residential	18.0	5.0	7.8	1.000000
5	106	airport	uptown	26.5	4.1	11.5	0.400000
6	107	suburbs	hotel	13.0	3.9	4.0	0.266667
7	108	downtown	downtown	16.0	4.7	5.5	0.800000
8	109	city center	station	16.0	4.5	6.5	0.666667
9	110	airport	residential	20.0	4.0	8.0	0.333333

Summary of cleaned dataset:

Justification:

Cleaning ride-sharing data improves operational efficiency and decision-making. Standardizing location names avoids duplication and confusion. Filling missing fare values ensures revenue analysis is accurate. Removing duplicate entries prevents incorrect records, and normalizing ratings helps in fair comparison. Overall, the cleaned dataset becomes reliable for business insights and analytics.